

Лабораторна робота №7

UX-інженерія: Проєктування Low-Fidelity Wireframes та архітектури технічної стійкості

Мета лабораторної роботи:

Навчитися трансформувати стратегічні інсайти продукту в системну архітектуру, що забезпечує стабільність інтерфейсу при зміні обсягів даних, роздільної здатності екранів та станів з'єднання. Оволодіти інженерним підходом до проєктування інтерфейсів через створення низькодеталізованих прототипів (Low-Fidelity Wireframes) та розробку супровідної технічної документації.

У результаті виконання роботи студент має навчитися:

- Проєктувати системну сітку: Налаштовувати Layout Grids та базову 8-піксельну модульну систему для забезпечення візуального ритму та точності верстки (Pixel Perfection).
- Створювати архітектуру компонентів: Збирати складні інтерфейсні блоки за допомогою вкладених Auto Layouts, імітуючи деревоподібну структуру реальної верстки (DOM/View Hierarchy).
- Забезпечувати адаптивність (Responsive/Adaptive Design): Проєктувати логіку перешикування контенту (Reflow) та налаштовувати правила масштабування елементів (Constraints & Resizing).
- Валідувати стійкість інтерфейсу: Проводити стрес-тестування макетів на переповнення динамічним контентом (Content Overflow) та проєктувати стратегію пріоритетного завантаження (Skeleton Loading).
- Формалізувати технічні вимоги: Розробляти анотовані схеми (Annotation Maps) із зазначенням типів даних, обов'язковості полів та зв'язку елементів із бізнес-логікою (JTBD).
- Комунікувати з розробниками: Використовувати професійний лексикон (Box Model, Breakpoints, Latency, Layout Shift) та стандарти іменування для ефективної передачі проєкту в імплементацію.

ПРАКТИЧНА ЧАСТИНА

Завдання 0. Технічні вимоги до проєктування (Design-for-Dev)

Згідно додатку до лаб.роботи провести кроки підготовчих дій:

- Крок 1. Налаштування Grid Layout Systems за типами платформ: Desktop, Web Mobile, Mobile App, Tablet App.
- Крок 2. Вимоги до стилістики (Low-Fi стандарти).
- Крок 3. Побудова композиції через вкладеність (Nested Auto Layouts).
- Крок 4. Налаштування адаптивної поведінки (Resizing).

Завдання 1. Розробка адаптивних Low-Fidelity Wireframes

На основі ієрархії сторінок (Sitemap із ЛРН№4) та сценаріїв взаємодії (User Flow із ЛРН№5) необхідно розробити скелетні макети (вайрфрейми) для індивідуального проєкту.

Вимога до іменування та організації:

Для інженера програмного забезпечення правильне іменування (Naming Convention) — це міст між дизайном і кодом. Коли Frontend-розробник бачить ваші макети, він має одразу зрозуміти, до якого медіа-запиту (@media query в CSS) або ресурсу (в Android/iOS) відноситься фрейм.

1. Всі фрейми мають бути чітко підписані за схемою: Платформа / Тип пристрою / Розмір (Ширина px) / Назва екрана.
2. Для кожного Breakpoint додайте технічну примітку (Annotation) із вказанням:
 - Ширини контентної області (Content width).
 - Параметрів сітки (Columns, Gutter, Margins).
3. Об'єднайте адаптивні версії однієї сторінки в Figma Section для зручної навігації в Dev Mode.

Такий підхід привчає до Scale-проектів: коли в проєкті 100+ екранів, назви на кшталт "Screen 1", "Screen 2 copy" призводять до хаосу в розробці.

Системний підхід до іменування

Замість тільки назв виду девайсів (наприклад, "iPhone 15") слід використовувати назви, що вказують на тип пристрою та ширину в пікселях. Використовуйте 'Snake_case' або 'Kebab-case' для іменування фреймів, та нижнє підкреслення або дефіси замість пробілів у назвах, що полегшує експорт активів розробникам.

Приклади для Web-платформи (Responsive):

- Web / Desktop / 1440+ / Home_Page (Базова версія для великих екранів).
- Web / Laptop / 1024 / Home_Page (Версія для планшетів у ландшафті або малих ноутбуків).
- Web / Tablet / 768 / Home_Page (Версія для iPad/планшетів).
- Web / Mobile / 360-390 / Home_Page (Мобільна версія).

Приклади для Mobile App (Adaptive):

- App / Mobile / Max (430px) / Dashboard
- App / Mobile / Base (390px) / Dashboard (Основний макет).
- App / Mobile / Compact (320px) / Dashboard (Критична точка для перевірки верстки).

Маркування станів (Annotation)

Поруч із фреймами на робочому просторі Figma (поза самими фреймами) варто розмішувати текстові мітки з технічними параметрами сітки, які допомагають Frontend-розробнику швидко налаштувати CSS-сітку:

- Breakpoint: min-width: 1200px.
- Grid: 12-col / 24px gutter.
- Margins: 80px.

Створення "Section" у Figma

Найкращий спосіб організувати брейкпоїнти — це використовувати інструмент Section (Shift+S).

- Назвіть секцію відповідно до сторінки: [Section] Головна сторінка.
- Всередині секції розмістіть фрейми зліва направо: від найбільшого (Desktop) до найменшого (Mobile).
- Це дозволяє розробнику в режимі Dev Mode одразу побачити всі варіації однієї сторінки.

Технічні аспекти індивідуальних варіантів для студента

Кожен студент обирає для реалізації один із наведених варіантів, з яким працюватиме впродовж наступних лаб.робіт.

Варіант А: Web Platform (Desktop & Web Mobile)

- Обсяг: 3–4 ключові сторінки для Desktop та їх відповідні адаптації для Web Mobile.
- Технічні параметри:
 - Desktop Grid: 12-колонкова центрована сітка (рекомендована ширина фрейма 1440px).

- Web Mobile Grid: 4-колонкова «гумова» сітка (рекомендована ширина фрейма 360–390px).
- Методологія (Responsive Web Design):
 - Використовуйте Auto Layout із налаштуванням Fill container для всіх секцій.
 - Навігаційна відповідність: для Desktop спроекуйте Header (горизонтальне меню), для Web Mobile — Hamburger Menu або Header (зверху).
- Вимога: Продемонструвати перешикування контенту (Reflow) — наприклад, перехід від 3–4 колонок карток на десктопі до 1 колонки на мобайлі.

Варіант Б: Mobile App (Smartphone & Tablet App)

- Обсяг: 3–4 ключові екрани для Mobile App та 2–3 пріоритетні екрани для Tablet App.
- Технічні параметри:
 - Mobile App Grid: 4-колонкова сітка з фіксованими боковими полями (Margins 16–20px).
 - Tablet App Grid: 8-колонкова сітка (рекомендовані пресети iPad або Android Tablet).
- Методологія (Adaptive/Native App Design):
 - Обов'язково зарезервуйте Safe Areas: зверху — Status Bar, знизу — Home Indicator (для iOS) або Navigation Bar (для Android).
 - Навігаційна відповідність: для Mobile App спроекуйте Tab Bar (нижню панель навігації). Для Tablet App дозволяється використання Side Rail (бокової панелі).
- Вимога: Використовуйте Constraints для фіксації системних елементів та Auto Layout для адаптації елементів керування під різну ширину екранів.

Примітка (для Варіанту Б):

При проектуванні за Варіантом Б (Mobile & Tablet App) студент повинен розмежувати сценарії використання залежно від ролі користувача. У професійній розробці планшети найчастіше використовуються як робочі інструменти для персоналу (POS-системи, адмін-панелі, моніторинг).

1. Mobile App (Смартфон): Проектується як клієнтський інтерфейс. Це екрани для звичайних користувачів додатком (User/Customer), де пріоритетом є швидкість доступу до функцій «на ходу», простота та ергономіка Thumb Zone.
2. Tablet App (Планшет): Проектується як інтерфейс для внутрішніх ролей (Staff/Internal Roles). Це екрани для ідентифікованих співробітників (адміністратор, менеджер, кур'єр, оператор складу тощо).
 - Фокус: Робота з великим обсягом даних, адміністрування, моніторинг замовлень або керування процесами.
 - Особливість: Використовуйте переваги більшої площі екрана для відображення детальних таблиць, бічних панелей (Sidebars) або складних форм редагування.

Завдання 2. Технічний стрес-тест та специфікація логіки адаптивності

Мета: Валідувати технічну стійкість макетів до змін в'юпорту та сформулювати «Reflow-специфікацію» — інструкцію для Frontend-розробника щодо перешикування компонентів.

Професійний макет у Figma — це не статична картинка, а набір інструкцій для браузера чи операційної системи. Для дизайнера важливо не просто «намалювати», а задекларувати математичну логіку поведінки елементів при зміні роздільної здатності. Це завдання імітує процес передачі верстки у розробку, де дизайнер пояснює, як працюватимуть властивості flex-grow, width:100% або @media queries.

Ця умова є логічним завершенням технічної документації макета та демонструє, що студент розуміє не лише візуальну частину, а й алгоритмічну логіку верстки, що є критично важливим для майбутньої комунікації з Frontend-розробниками.

Крок 2.1. Проведення Responsive Stress-test (Практична перевірка)

Після завершення розробки вайрфреймів студент повинен провести технічний стрес-тест створених макетів, аби довести, що налаштування Auto Layout та Constraints виконані коректно і макет адаптується до змін в'юпорту без деформації елементів.

Перевірка проводиться відповідно до обраного варіанта проєкту.

1) Загальна вимога для обох варіантів: Під час тестування текст не повинен «обрізатися» або «вилазити» за межі батьківського контейнера, а іконки та зображення мають зберігати свої пропорції (Aspect Ratio).

2) Варіант А: Web Platform (Desktop & Web Mobile)

- Тест на «гумовість» (Fluidity): Виберіть головну сторінку Web Mobile. При горизонтальному розтягуванні фрейма від 320px до 450px контент (картки, кнопки, текстові блоки) повинен автоматично заповнювати ширину (Fill container), зберігаючи сталі бокові відступи (Margins: 16px).
- Тест на трансформацію (Reflow): Продемонструйте, як десктопний макет (1440px) реагує на звуження до Tablet-розміру (768px). Елементи, що не вміщаються по горизонталі, мають автоматично переноситися на наступний рядок (Wrap) або змінювати свою структуру (наприклад, горизонтальне меню ховається в «бургер»).
- Результат: 2-3 стани одного екрана, що демонструють поведінку контенту на різних медіа-запитах.

3) Варіант Б: Mobile App (Smartphone & Tablet App)

- Тест на відповідність Safe Areas: Макет смартфона повинен коректно відображатися при зміні ширини фрейма від 320px до 430px. Системні елементи (Status Bar та Home Indicator) мають залишатися на своїх місцях (Constraints), а інтерактивні елементи (кнопки, інпути) — розтягуватися, не перекриваючи «безпечні зони».
- Тест на орієнтацію Tablet App: Змініть орієнтацію планшетного макета з портретної (Portrait) на ландшафтну (Landscape). Кількість колонок у списках або сітках карток має автоматично збільшитися (наприклад, з 2-х до 3-х або 4-х), раціонально заповнюючи додатковий простір.
- Результат: Демонстрація адаптивності інтерфейсу під різні форм-фактори та зміну орієнтації екрана.

Крок 2.2. Специфікація стратегії перешикування (Reflow Strategy Table)

За результатами стрес-тесту необхідно скласти технічну таблицю Reflow Strategy для 3-х ключових компонентів системи. Ця таблиця слугує інструкцією для Frontend-розробника, описуючи, як саме змінюються параметри елементів при переході між брейкпоінтами.

Варіант А: Web Platform (Desktop vs Web Mobile)

Акцент на зміні властивостей CSS (Width, Flex-direction) та поведінці контентної сітки.

Компонент	Desktop (1440px)	Web Mobile (360-390px)	Breakpoint / Event
Header (Навігація)	Горизонтальний список посилань (flex-direction: row)	Приховане меню-бургер (display: none для списку)	@media max-width: 768px
Grid (Сітка карток)	3 або 4 колонки (grid-template-columns: 1fr 1fr...)	1 колонка (картки на всю ширину екрана)	@media max-width: 1024px
Hero Section (Банер)	Текст зліва, зображення справа. Висота фіксована.	Текст над зображенням. Висота залежить від контенту (Hug).	@media max-width: 600px

Варіант Б: Mobile App (Smartphone vs Tablet App)

Акцент на зміні паттернів навігації та використанні додаткової площі екрана планшета.

Компонент	Smartphone (390px)	Tablet (744-834px)	Breakpoint / Event
Main Navigation	Tab Bar (Нижня панель з іконками)	Side Rail (Бокова вертикальна панель)	Зміна форм-фактора пристрою
Dashboard Card	Вертикальна орієнтація. Ширина Fill Container.	Горизонтальна орієнтація (зображення + детальний опис).	Перехід у ландшафтну орієнтацію
Content Area	Одна колонка контенту. Скрол зверху вниз.	Master-Detail View (Список зліва, деталі обраного об'єкта справа).	Ширина в'юпорту > 600px

Вимоги до заповнення таблиці:

- Компонент: Вкажіть назву об'єкта, який проходить трансформацію.
- Технічний опис: Не пишіть «стає меншим», пишіть інженерною мовою: Зміна з Horizontal Auto Layout на Vertical», Width:100% (Fill)», Visible:False.
- Breakpoint / Event: Вкажіть точне значення ширини (для Web) або умову (для App), при якій спрацює зміна логіки.

Інженерна цінність: Ця специфікація дозволяє студенту-інженеру чітко розділити «візуальну зміну» та «технічну реалізацію». Вона є основою для майбутньої верстки інтерфейсу, де кожен рядок таблиці перетворюється на конкретну умову в коді.

Крок 2.3. Математичне обґрунтування (Resizing & Constraints)

Для завершення Завдання 2 необхідно формалізувати правила масштабування елементів. У цьому кроці студенти мають довести, що вони розуміють різницю між відносною версткою (Web) та прив'язкою до системних осей (Native Apps).

Умова: Використовуючи анотації (Sticky notes), опишіть налаштування внутрішньої логіки компонентів. Студент має обґрунтувати вибір параметрів масштабування, виходячи зі специфіки своєї платформи.

Варіант А: Web Platform (Логіка Flexbox та Box Model)

Акцент на тому, як контент поводить себе при зміні розміру вікна браузера. Студент має вказати:

- Fluid Width (Fill Container): Які текстові блоки та контейнери мають «гумову» ширину (займають весь доступний простір), щоб адаптуватися до будь-якого монітора.
- Fixed Width/Height (Фіксовані): Які текстові блоки та контейнери використовують сталі розміри, щоб вони не деформувалися при «гумовій» верстці.
- Min/Max Constraints: Вказати межі, за якими контент перестає стискатися, щоб уникнути накладання елементів.
- Hug Contents (Залежні від контенту): Вкажіть елементи (кнопки, теги), розмір яких підлаштовується під внутрішній текст або об'єкт, не створюючи зайвого порожнього простору.
- Constraints: Поясніть логіку прив'язки елементів, що знаходяться поза Auto Layout (наприклад, Floating Action Button, закріплена до Bottom & Right).
- Alignment: Логіка вирівнювання всередині секцій (напр. для шапки сайту, щоб логотип та меню завжди були біля протилежних країв).

Варіант Б: Mobile App (Логіка Anchoring та Safe Areas)

Акцент на фіксації елементів відносно осей пристрою та системних зон. Студент має вказати:

- Sticky Elements (Constraints): Як зафіксовані елементи (наприклад, кнопка дії зафіксована до Bottom-Right, а пошуковий рядок до Top-Stretch).
- Aspect Ratio Preservation: Які елементи (іконки, фото) мають фіксовані розміри або пропорції, щоб не деформуватися на різних екранах (від iPhone Mini до iPad).

- **Safe Area Offsets:** Обґрунтування відступів від країв екрана, щоб контент не перекривався «чубчиком» (Notch) або системним індикатором перемикачів додатків.

Приклад опису (Justification):

- Центральна картка налаштована на Fill Container, тому на екранах різної ширини (від 320px до 430px) вона розтягується автоматично, зберігаючи константні бокові відступи (Margins) у 16px.
- Бокова навігаційна панель (Side Rail) має Fixed Width (80px), щоб при збільшенні екрана робоча область контенту розширювалася, а меню залишалося незмінним.

Вимоги до оформлення (для обох варіантів):

1. Оберіть один складний компонент (наприклад, картку з кнопками та текстом).
2. За допомогою стрілок або виносних ліній у Figma покажіть:
 - Які елементи мають налаштування Hug Contents (розмір залежить від довжини тексту).
 - Які елементи мають налаштування Fill Container (займають весь вільний простір).
 - Для Варіанта Б покажіть налаштування Constraints (сині пунктирні лінії у Figma).

Інженерна цінність: Цей крок вчить студентів думати категоріями Auto Layout. Розуміння параметрів Hug/Fill є критичним для Frontend-розробника, оскільки це пряма аналогія до властивостей flex-basis, flex-grow та flex-shrink у коді.

Завдання 3. Проєктування динамічних станів даних (UI States)

Мета: Навчитися проєктувати інтерфейс, який залишається візуально стабільним під час очікування даних, та мінімізувати негативний вплив затримок мережі (Latency) на користувацький досвід.

Крок 3.1. Анатомія Skeleton Screens (Графічна специфікація)

На цьому етапі ви створюєте візуальний «зліпок» майбутнього контенту. Під час виконання асинхронних запитів до сервера користувач не повинен бачити порожній екран або статичний Spinner (якщо очікування триває <3 сек). Ви маєте спроектувати систему інертних заглушок, які створюють ілюзію миттєвого завантаження та зберігають структуру екрана до моменту рендерингу даних. Це завдання демонструє зв'язок між часом відповіді сервера (Latency) та візуальним станом фронтенду.

Умова: Для одного найбільш динамічного екрана вашого проєкту (наприклад, стрічка новин, список товарів або профіль користувача) розробити статичний макет-заклушку (Skeleton Screen).

Технічні вимоги:

- **Геометрична абстракція:** Заборонено використовувати реальні іконки, текст або логотипи. Використовуйте виключно абстрактні форми (Placeholder shapes): прямокутники з округленими кутами для тексту та кола для аватарок/іконок. Колірна гама — світло-сірі відтінки.
- **Layout Parity (Ідентичність сітки):** Скелетони мають базуватися на тих самих налаштуваннях Auto Layout (Padding, Gap, Alignment), що й основний макет.
- **Hierarchy Preservation:** Збережіть візуальну ієрархію контенту. Заклушки для заголовків мають бути масивнішими (товщина/ширина) за основний текст, навіть у стані завантаження.

Критерій успіху: При накладанні Skeleton-макета на фінальний Low-Fi макет у Figma (Opacity 50%), межі заглушок мають збігатися з межами реальних елементів з точністю до 1px.

Крок 3.2. Стратегія пріоритетного завантаження (Skeleton Loading Strategy)

Це завдання фокусується на продуктивності інтерфейсу та стабільності верстки (Cumulative Layout Shift / CLS – ситуації, коли завантажений об'єкт «штовхає» інші елементи) і поєднує UX-дизайн із мережевою логікою та рендерингом.

У реальних умовах дані з API приходять не одночасно: статичні ресурси вантажаться миттєво, легкі текстові дані — швидше, а «важкі» графічні об'єкти або результати складних обчислень — із затримкою. Для дизайнера важливо розробити стратегію завантаження, яка створює ілюзію швидкодії системи. Замість того, щоб чекати повного завантаження сторінки, ми використовуємо Skeleton Screens із різним часом появи, забезпечуючи візуальну стабільність інтерфейсу. На цьому етапі ви проєктуєте черговість появи елементів, щоб уникнути «стрибків / тремтіння» контенту (Layout Shift) при поетапному рендерингу (у момент заміни скелетона реальними даними після отримання відповіді від API).

Умова: Для екрану із кроку 3.1 скласти специфікацію «Anti-Jank» (Схему пріоритетного завантаження), розділивши елементи екрана на три рівні пріоритетності за часом рендерингу. Студент має класифікувати елементи екрана за трьома часовими категоріями:

- Level 0: Instant Render (0 мс): Статичні елементи, що зашиті в клієнтську частину (системний Status Bar, Header з назвою розділу, нижня панель навігації (Tab Bar), фонові підкладка екрана тощо). Вони з'являються миттєво та не залежать від API.
- Level 1: Lite Data (100–200 мс): Легкі текстові дані або метадані з БД (імена, назви, статуси). На екрані вони відображаються статичними сірими заглушками.
- Level 2: Heavy Assets (300–800+ мс): Важкі ресурси або дані від сторонніх сервісів (фото, карти, графіки). Для них використовуються анімовані скелетони з ефектом «пульсації» (Shimmer effect).

Технічна вимога:

- Результат подається у вигляді анотованого макета, де для кожного «важкого» елемента (Level 2) за допомогою часових міток, вказана послідовність «проявлення» контенту (спосіб резервування місця). Студент має обґрунтувати, як він уникає Layout Shift (ситуація, коли після завантаження картини текст раптово «падає» нижче).

Результат виконання:

Панель у Figma, що демонструє три стани одного екрана поруч: Initial State → Loading State → Final Content.

- Initial State (тільки Level 0).
- Loading State (Skeleton Screens Level 1 + Level 2).
- Final Content (повністю завантажені дані).

Жоден елемент не повинен змінювати свої координати (X, Y) під час переходу між цими трьома станами.

Інженерна цінність: Це завдання привчає студентів до концепції Optimistic UI та правильного використання властивостей min-height та aspect-ratio у верстці, що є критичним для проходження тестів продуктивності (Google Core Web Vitals).

Приклад обґрунтування для кейсу UniMate:

Щоб уникнути зміщення контенту на екрані розкладу, ми заздалегідь резервуємо контейнер під схему поверху (Skeleton 2) із фіксованим Aspect Ratio. Навіть якщо текст пари (Skeleton 1) прийде швидше, він не підніметься вгору, оскільки місце під мапу вже зарезервоване завантажувальною заглушкою ідентичного розміру.

Контрольна таблиця пріоритетів:

Шар завантаження	Елементи інтерфейсу	Технічна причина затримки	UX-відгук (Placeholder)
Instant	Tab Bar, Header, BG	Static Asset (Local)	Миттєвий показ (0ms)
Level A	Room Number, Subject Name	JSON Response (API)	Static Grey Box
Level B	Indoor Map, Avatar Photo	Heavy Data / Asset Load	Shimmer Animation

Крок 3.3. Проектування логіки «Довгого контенту» (Content Truncation Strategy)

Мета: Навчитися проектувати інтерфейси, що зберігають візуальну цілісність при отриманні динамічних даних непередбачуваної довжини. Оволодіти методами обробки переповнення контенту (Overflow) та розробити стратегію обрізки тексту (Truncation).

Інтерфейс має залишатися цілісним незалежно від обсягу вхідних даних. Студент повинен продемонструвати інженерний підхід до обробки «нестандартного» контенту, щоб уникнути накладання елементів або руйнування сітки. Якщо не спроектувати поведінку блоків заздалегідь, довгий текст «розвалить» верстку або перекриє сусідні елементи. У цьому завданні ви повинні розробити правила, за якими ваша система оброблятиме «незручні» дані.

Умова: Оберіть один компонент (наприклад, картку товару, елемент списку або профіль користувача) та продемонструйте три технічні сценарії обробки надлишкового тексту.

Проектування стратегій обробки (Overflow Patterns):

1) Сценарій А: Фіксована висота (Line Clamp / Ellipsis). Використовується для збереження жорсткої сітки (наприклад, у каталогах).

- Дія: Введіть текст, що значно перевищує розмір контейнера (наприклад, назва об'єкта на 4-5 рядків).
- Реалізація у Figma: Налаштуйте обрізання тексту через "Truncate text" (трикрапка).
- Результат: Текст обрізається, а Auto Layout зберігає сталу (Fixed) висоту всього компонента.

2) Сценарій Б: Динамічне розширення (Auto-expand). Використовується для детальних описів, де важливо показати весь текст.

- Дія: Введіть довгі дані (наприклад, подвійне прізвище або складний заголовок).
- Реалізація у Figma: Встановіть для текстового шару та його батьківського контейнера налаштування Vertical Resizing: Hug contents.
- Результат: При збільшенні кількості рядків контейнер автоматично розширюється по вертикалі, «відштовхуючи» нижні елементи без втрати внутрішніх відступів (Gaps).

3) Сценарій В: Горизонтальна прокрутка (Horizontal Scroll / Container Overflow). Використовується для списків категорій, тегів або фільтрів, які мають залишатися в один рядок, не переважуючи вертикальний простір.

- Дія: Створіть ряд елементів (наприклад, 8-10 кнопок-фільтрів), сумарна ширина яких значно перевищує ширину екрана смартфона.
- Реалізація у Figma: Об'єднайте елементи в Auto Layout (напрямок: Horizontal). Помістіть цей довгий список у батьківський фрейм (Container), ширина якого дорівнює ширині макета (напр., 390px). Увімкніть налаштування Clip content у правій панелі Design. Щоб підказати користувачу (і розробнику) про наявність скролу, налаштуйте розташування елементів так, щоб останній видимий елемент був обрізаний краєм екрана наполовину. Це візуальний паттерн, що сигналізує про продовження контенту.
- Результат: Частина елементів стає невидимою (обрізаною), але вони фізично залишаються в структурі шарів. Це моделює стан «за межами в'юпорту».
- Технічна анотація: Студент має додати текстову помітку поруч із макетом: «Container Overflow: Hidden. Елементи за межами фрейма мають бути доступні через Horizontal Scroll (імплементация через overflow-x: auto)».

Інженерна вимога: Студент має продемонструвати ці стани поруч (Original vs Overflow), щоб довести:

- Елементи не виходять за межі батьківського фрейма.
- Текст не перекриває кнопки або іконки.
- Вертикальний ритм (8px) зберігається навіть при деформації блоку.

Інженерний акцент: Ці три сценарії дають студенту повне розуміння властивості Overflow у CSS:

- Line Clamp – це робота з text-overflow: ellipsis.
- Auto-expand – це робота з height: auto.
- Horizontal Scroll – це робота з overflow-x: scroll.

Завдання 4. Анатомія структури (Annotation Map)

Гарний дизайн не потребує пояснень користувачу, але завжди потребує пояснень розробнику. Ваші анотації – це декларація намірів. Коли ви чітко розділяєте статичний текст і динамічні дані, ви вже на 50% проєктуєте архітектуру вашої майбутньої бази даних та API-запитів.

Крок 4.1. Специфікація типів даних (Data Architecture Map)

Професійний UX-дизайн – це не лише візуалізація, а й проєктування інформаційної моделі. Студент повинен продемонструвати розуміння того, які елементи інтерфейсу є статичними («зашитими» в код), а які – динамічними (отриманими з бази даних через API). Це завдання є ідеальним перехідним містком від дизайну до системної архітектури, та привчає студента бачити за візуальним інтерфейсом структуру даних.

Умова: Оберіть один найбільш функціональний екран вашого проєкту (наприклад, Картка товару, Профіль користувача або Список замовлень) та створіть схему походження даних.

Технічна вимога:

- Класифікація контенту: Використовуйте кольорові маркери або виноски (Annotations) у Figma, щоб розділити елементи на дві групи:
 - Dynamic Data (Динамічні дані): Дані, що завантажуються з БД/API (наприклад, прізвище користувача, статус замовлення, ціна). Вимога: Для кожного динамічного елемента вкажіть очікуваний тип даних (String, Boolean, Number/Integer, Date, Image URL). Вкажіть, чи є поле обов'язковим (Required) чи може бути пустим (Nullable) — це визначить логіку відображення екрана розробником.
Приклад: Ім'я користувача (String), Баланс рахунку (Number), Статус замовлення (String/Enum).
 - Static Elements (Статичні елементи): Контент (текст та графіка), що є частиною інтерфейсу і не змінюється залежно від даних у БД.
Приклад: Назви кнопок («Зберегти»), заголовки секцій («Особиста інформація»), плейсхолдери полів вводу, логотипи, лейбли полів, іконки категорій тощо.

Інженерна цінність: Це допомагає Frontend-розробнику одразу зрозуміти структуру майбутнього JSON-об'єкта, який мав би прийти від Backend для рендерингу цього екрана.

Крок 4.2. Пріоритезація за JTBD (Аналіз інформаційної ієрархії)

Розташування контенту на екрані має бути результатом аналізу пріоритетів користувача. Студент повинен обґрунтувати архітектуру екрана, спираючись на концепцію Jobs to be Done (JTBD), опрацьовану в ЛР№3. Це вчить студентів-інженерів приймати дизайнерські рішення не на основі «мені так подобається», а на основі об'єктивних потреб користувача та бізнес-логіки.

Умова: Для одного ключового екрана вашого проєкту створіть анотовану схему пріоритетів, яка пояснює вертикальну ієрархію блоків.

Технічні вимоги:

- Зв'язок із JTBD: Виберіть головну "роботу" (Job), яку виконує користувач на цьому екрані.

- Обґрунтування ієрархії (Justification): Використовуючи виноски (Annotations), поясніть позицію кожного великого блоку контенту.
 - Чому цей блок у верхній третині (Above the fold)?
 - Чому цей елемент є найбільшим (Visual Weight)?

Приклад аргументації:

- Блок 'Поточна пара' знаходиться у верхній третині екрана, оскільки згідно з JTBD студента Максима, його головна мета — миттєво зорієнтуватися в часі та кабінеті при вході в корпус, не вдаючись до скролу.
- Кнопка 'Повторити замовлення' винесена у фокусну зону, оскільки основна Job користувача — замовити звичний обід за 2 кліки під час короткої перерви.

UX-логіка: Студент має довести, що найважливіша інформація для виконання "роботи" знаходиться в зоні першого візуального контакту. Все, що є другорядним або допоміжним, має бути зміщено нижче по ієрархії.